

Condition Loops on Terraform

To Begin with the Lab

Summary of the Lab

In this lab, we learn how to use conditional statements in Terraform to control infrastructure creation based on variable values. Instead of manually changing the number of resources, Terraform automatically decides how many EC2 instances to create depending on the selected environment. If the environment is Development, only one EC2 instance is launched, while Production creates two instances. The lab demonstrates how to define variables, use a conditional (ternary) expression, deploy the infrastructure, verify the created resources in AWS, and finally remove them using Terraform. This approach makes infrastructure more flexible, reusable, and easier to manage across different environments.

- Create a new Terraform project for conditional statements.
- Create the `create_instance.tf` file to define the AWS provider and EC2 module.
- Create the `variable.tf` file to define the AWS region and environment variables.
- Use a conditional (ternary) expression to determine the number of EC2 instances:
- If `environment == "Production"` → create 2 instances.
- Otherwise → create 1 instance.
- Initialize Terraform using `terraform init`.
- Run `terraform plan` to preview the resources.
- Apply the configuration using `terraform apply`.
- Verify the EC2 instance(s) in the AWS Management Console.
- Destroy the resources using `terraform destroy`.
- Change the environment variable from Development to Production.
- Run `terraform plan` and `terraform apply` again to verify that two EC2 instances are created.
- Confirm the new resources and their tags in AWS.
- Clean up the infrastructure using `terraform destroy`.

Terraform Built-In Function

To Begin with the Lab

Summary of the Lab

Open the Terraform project directory and launch the interactive Terraform Console, then evaluate basic string expressions and practice using functions including `base64encode()`, `base64decode()`, `chunklist()`, `coalesce()`, `compact()`, `concat()`, `contains()`, `element()`, `uuid()`, `timestamp()`, `length()`, `split()`, and `replace()` to understand Terraform's built-in function capabilities.

Step	Function	Example	Output
1	Test a string	"Hello, Terraform"	"Hello, Terraform"
2	<code>base64encode()</code>	<code>base64encode("Terraform_for_Deployment_Automation")</code>	"VGVycmFmb3JtX2Zvc19EZXBsb3ltZW50X0F1dG9tYXRpb24="
3	<code>base64decode()</code>	<code>base64decode("VGVycmFmb3JtX2Zvc19EZXBsb3ltZW50X0F1dG9tYXRpb24=")</code>	"Terraform_for_Deployment_Automation"
4	<code>chunklist()</code>	<code>chunklist(["item1","item2","item3","item4","item5"],2)</code>	[[["item1","item2"],["item3","item4"],["item5"]]]
5	<code>coalesce()</code>	<code>coalesce("", "", "Terraform")</code>	"Terraform"
6	<code>compact()</code>	<code>compact(["item1","", "item2","", "item3"])</code>	["item1","item2","item3"]
7	<code>concat()</code>	<code>concat(["item1","item2"],["item3","item4"],["item5"])</code>	["item1","item2","item3","item4","item5"]
8	<code>contains()</code>	<code>contains(["item1","item2","item3"], "item2")</code>	true
		<code>contains(["item1","item2","item3"], "item4")</code>	false
9	<code>element()</code>	<code>element(["item1","item2","item3"], 1)</code>	"item2"
		<code>element(["item1","item2","item3"], 5)</code>	"item3" (Terraform wraps around the list when the index is out of range.)
10	<code>uuid()</code>	<code>uuid()</code>	"9d29b4c8-3dc8-49c4-9c62-dc2a8b1c7d6d" (random every time)

Step	Function	Example	Output
11	timestamp()	timestamp()	"2026-06-25T12:34:56Z" (<i>current UTC time</i>)
12	length()	length(["item1","item2","item3"])	3
		length("Terraform")	9
13	split()	split(",", "USA,Canada,India")	["USA","Canada","India"]
14	replace()	replace("This is India","i","!")	"Th!s !s Ind!a"

```
root@ip-172-31-17-1:~/workspace# terraform console
> "Hi, I am using Terraform"
"Hi, I am using Terraform"
> base64encode("Terraform_for_Deployment_Automation")
"VGVyYcmFmb3JtX2Zvc19EZXBsb3ltZW50X0F1dG9tYXRpb24="
> base64decode("VGVyYcmFmb3JtX2Zvc19EZXBsb3ltZW50X0F1dG9tYXRpb24=")
"Terraform_for_Deployment_Automation"
> chunklist(["item1","item2","item3","item4","item5"],2)
tolist([
  tolist([
    "item1",
    "item2",
  ]),
  tolist([
    "item3",
    "item4",
  ]),
  tolist([
    "item5",
  ]),
])
> coalesce("", "", "Terraform")
"Terraform"
> compact(["item1","", "item2","", "item3"])
tolist([
  "item1",
  "item2",
  "item3",
])
> concat(["item1","item2"],["item3","item4"],["item5"])
[
  "item1",
  "item2",
  "item3",
  "item4",
  "item5",
]
> contains(["item1","item2","item3"],"item2")
true
> contains(["item1","item2","item3"],"item4")
false
> contains(["item1","item2","item3"],"item")
false
> |
```

```
> contains(["item1","item2","item3"],"item2")
true
> contains(["item1","item2","item3"],"item4")
false
> contains(["item1","item2","item3"],"item")
false
> element(["item1","item2","item3"],1)
"item2"
> element(["item1","item2","item3"],5)
"item3"
> uuid()
"d38cca12-5afb-ded2-8fb9-75b75517a31f"
> timestamp()
"2026-06-25T17:22:25Z"
> length("Terraform")
9
> |
```

```
> length(["item1","item2","item3"])
3
> |
```

```
> split(",", "USA,Canada,India")
tolist([
  "USA",
  "Canada",
  "India",
])
> replace("This is India","i","!")
"Th!s !s Ind!a"
> |
```

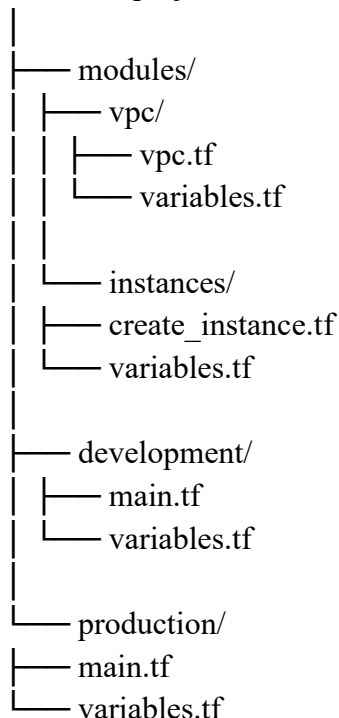
Terraform Project Structure

To Begin with the Lab

Summary of the Lab

In this lab, we learn how to organize a production-ready Terraform project by separating reusable modules from environment-specific configurations. Instead of writing duplicate code, we create reusable VPC and EC2 Instance modules and use them in separate Development and Production environments. Each environment has its own variables, allowing the same infrastructure to be deployed with different settings such as AWS Region and Environment Name. This modular approach makes the infrastructure easier to maintain, reuse, and scale while following industry best practices.

terraform-project-structure/



- Create a production-style Terraform project structure.
- Create separate directories for:
 - modules
 - development
 - production
- Build a reusable VPC module.
- Use the official Terraform AWS VPC module from the Terraform Registry.
- Configure:
 - VPC
 - Public Subnets
 - Private Subnets
 - Internet Gateway

- NAT Gateway
 - Route Tables
- Define output values such as:
 - VPC ID
 - Public Subnets
 - Private Subnets
- Create a reusable EC2 Instance module.
- Configure:
 - Key Pair
 - Security Group
 - EC2 Instance
- Define variables required by the modules.
- Create the Development environment configuration.
- Create the Production environment configuration.
- Pass outputs from the VPC module as inputs to the EC2 module.
- Initialize Terraform.
- Deploy the Development environment.
- Verify all created resources in AWS.
- Deploy the Production environment using the same modules but different variable values.
- Verify the Production resources.
- Destroy all created resources after completing the lab.
- Built a reusable VPC module to provision a custom VPC, public/private subnets, Internet Gateway, NAT Gateway, and route tables, with outputs for the VPC ID and subnet IDs.
- Created an EC2 module that provisions a key pair, security group, and EC2 instance using the VPC module outputs for deployment within the custom VPC.
- Configured the Development environment (us-east-1) to deploy the complete infrastructure, including the custom VPC, networking components, security group, and one EC2 instance.
- Configured the Production environment (us-east-2) to deploy the same infrastructure with environment-specific variables after correcting the Availability Zone configuration.
- Terraform plans to create 25 resources:
- Plan: 25 to add, 0 to change, 0 to destroy.
- These resources include:
 - 1 Custom VPC
 - 3 Public Subnets
 - 3 Private Subnets
 - 1 Internet Gateway
 - 4 Route Tables (1 public + 3 private)
 - 6 Route Table Associations
 - 1 Security Group
 - 1 EC2 Key Pair
 - 1 EC2 Instance

- Default VPC resources (Default Security Group, Default Route Table, Default Network ACL)

```
    }  
  }  
}  
Plan: 25 to add, 0 to change, 0 to destroy.  
  
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```